

32 LinkedList Add “1” to a number represented as a Linked List.

```
class Solution {
public:
    // Function to reverse a linked list
    Node* reverse(Node* head) {
        Node* prev = NULL;
        Node* curr = head;
        Node* next = NULL
        while (curr != NULL) {
            next = curr->next;
            curr->next = prev;
            prev = curr;
            curr = next;
        }
        return prev;
    }
    Node* addOne(Node* head) {
        // Step 1: Reverse the list
        head = reverse(head);
        // Step 2: Add 1
        Node* curr = head;
        int carry = 1; // because we want to add 1
        while (curr != NULL && carry) {
            int sum = curr->data + carry;
            curr->data = sum % 10;
            carry = sum / 10;
            if (carry && curr->next == NULL) {
                curr->next = new Node(0); // extend list if needed
            }
            curr = curr->next;
        }
        // Step 3: Reverse the list back
        head = reverse(head);
        return head;
    }
}
```

The screenshot displays a coding platform interface with a dark theme. On the left, the 'Output Window' shows 'Compilation Results' and a 'Problem Solved Successfully' message. Below this, statistics are listed: 'Test Cases Passed: 1116 / 1116', 'Attempts: Correct / Total: 1 / 1', 'Accuracy: 100%', 'Points Scored: 4 / 4', and 'Time Taken: 0.11'. At the bottom, there are buttons for 'Solve Next' and 'Stay Ahead With:' followed by links to 'Pairwise swap elements of a linked list', 'Kth from End of Linked List', and 'Find length of Loop'.

The main editor on the right shows the C++ code for the solution, with line numbers 4 through 43. The code implements a three-step process: reversing the linked list, adding 1 with carry, and reversing it back. The code is as follows:

```
4 Node* curr = head;
5 Node* next = NULL;
6
7 while (curr != NULL) {
8     next = curr->next;
9     curr->next = prev;
10    prev = curr;
11    curr = next;
12 }
13 return prev;
14
15 Node* addOne(Node* head) {
16     // Step 1: Reverse the list
17     head = reverse(head);
18
19     // Step 2: Add 1
20     Node* curr = head;
21     int carry = 1; // because we want to add 1
22
23     while (curr != NULL && carry) {
24         int sum = curr->data + carry;
25         curr->data = sum % 10;
26         carry = sum / 10;
27
28         if (carry && curr->next == NULL) {
29             curr->next = new Node(0); // extend list if needed
30         }
31         curr = curr->next;
32     }
33
34     // Step 3: Reverse the list back
35     head = reverse(head);
36     return head;
37 }
38
39
40
41
42
43 }
```